Data Engineering: PLANIT 2.0 Technical Documentation

PLANIT 2.0 Technical Documentation

- Planit 2.0 API data extraction
  - Planit 2.0 API requests reference
- API implementation details
- Transformation and Load
- Useful References
  - Step functions reference
  - Load production and staging schemas reference
  - Client Database table names/prefix
  - How to backfill
  - Gitlab Code repository reference

Planit 2.0 API data extraction

Planit 2.0 API is exposed as RESTFUL endpoints secured using oauth2 authentication.

Client Id and secret are provided by the “planit” team. To get an access token to authorize endpoint calls, we exchange client id and secret, for the temporary access token using the provided authentication endpoint and grant_type of “client_credentials”, via a POST request.

Planit 2.0 API requests reference

- **Get Access Token**

Get temporary access token by exchanging client credentials.

Method	POST
Endpoint	**https://omg-planit-master.auth.eu-west-1.amazoncognito.com**
grant_type	client_credentials
client_id	Provided by annalect planit team
client_secret	Provided by annalect planit team

Authorisation headers JSON

```
{
    "Authorization": "Basic  <CLIENT_SECRET>",
    "Content-Type": "application/x-www-form-urlencoded"
}

```

Authorisation Request Body

```
{
    "grant_type": "client_credentials",
    "client_id":  "<CLIENT_ID>"
}
```

Response Body

```
{
   "access_token": "<ACCESS_TOKEN>",
    "expires_in": 3600,
    "token_type": "Bearer"
}
```

- **List Reports**

We can use the list endpoint to view a full list of views exposed via the planit 2.0 API, in order to get report IDs for relevant views.

Method	GET
Endpoint	https://www.onerollup.com/LivePivotsWebApi/api/v1/queries

Authorisation headers JSON

```
{
    "Authorization": "Bearer  <DACCESS_TOKEN>"
}
```

Example Response

```
{'queryList': [{'id': 305,
   'name': 'Test Flowchart Query',
   'description': 'Test across all brands',
   'category': 'Default',
   'modifiedWhen': '2022-08-16T07:01:15',
   'modifiedBy': 'test@test.com',
   'createdWhen': '2022-08-03T06:38:31',
   'createdBy': 'test@test.com'},
  .....
```

- **Get Report Data**

Get individual reports data.

Method	GET
Endpoint	https://www.onerollup.com/LivePivotsWebApi/api/v1/queries/<REPORT_ID>/data

Authorisation headers JSON

```
{
    "Authorization": "Bearer  <DACCESS_TOKEN>"
}
```

Example Response

```
{'layout': {'fields': [{'fieldName': 'AV Pre-Bid Cost (EUR)',
    'dataType': 'string',
    'currency': 'EUR'},
   {'fieldName': 'AV Pre-Bid Cost (LC)', 'dataType': 'float', 'currency': ''},
....
 'data': [
   {'AV Pre-Bid Cost (EUR)': '0',
```

```
   'AV Pre-Bid Cost (LC)': '0',
....
```

API implementation details

We receive a single file per report containing metrics and detail. In order to create fact and dimension from this single file we generate dimensions by creating unique dimension IDs for deterministic dimension fields, and append these to report rows.

Report API Transformation Implementation

- Create unique id for dims
  - This is performed by hashing all dim identified fields in the record.
- Group records weekly by the supplied date field (`bdcst_wk`).
- Load partitioned data to s3 bucket.

Load Partitioned Data to s3

Once the dimensions are identified and generated, we load the “transformed” report partition by **year/month/day** to the appropriate s3 bucket for the vendor.

Bucket References:

- Audi: `s3://omg-phd-audi-drive-data-stg/data/planit_v3/`
- Porsche: `s3://omg-phd-porsche-data-stg/planit_v2/`
- VW: `s3://omg-phd-vw-newcar-stg/data/data/planit_v3/`

Transformation and Load

Transformation and Load stages are implemented using the pipeline framework.

Types of Reports – Generated Fact/Dimensions

- Tactical
  - Dimension Tables
    - Date
    - Market
    - Flight
- Schematic
  - Dimension Tables
    - Date
    - Market

Useful References

Step functions reference

- **VW**:
  - **Extract**: omg-phd-vw-planit-v2-api
  - **Transform**: omg-phd-vw-pipeline-planit-new-transform

    - **Load**: omg-phd-vw-pipeline-planit-new-load
- **Audi**:
    - **Extract**: omg-phd-audi-planit-v2-api
    - **Transform**: omg-phd-audi-pipeline-planit-new-transform
    - **Load**: omg-phd-audi-pipeline-planit-new-load
- **Porsche**:
    - **Extract**: omg-phd-porsche-planit-v2-api
    - **Transform**: omg-phd-porsche-pipeline-planit-new-transform

Load production and staging schemas reference

- **VW**:
    - **production**: vwb
    - **schematic**: vw_planit_v3_sch
    - **tactical**: vw_planit_v3_tac
- **Audi**:
    - **production**: audi
    - **schematic**: audi_planit_v3_sch
    - **tactical**: audi_planit_v3_tac

Client Database table names/prefix

- **AUDI**:
    - **Fact**:
        - fact_ptc_tactical
        - fact_psc_schematic
    - **Dim prefixes**:
        - dim_ptc* (tactical)
        - dim_psc* (schematic)
- **VW**:
    - **Fact**:
        - fact_plt_tactical
        - fact_pls_schematic
    - **Dim prefixes**:
        - dim_plt* (tactical)
        - dim_pls* (schematic)
- **Porsche**: N/A (s3 datalake share)

How to backfill

Extract (API)

In order to back fill data to staging s3 using API pipeline,

- We use the report ID of the view containing the data. We can retrieve this report ID using the List report views API (ref above).

- Specify the report IDs as part of the pipeline step function parameters, e.g

```
"report_ids", [{
    "schematic": 290, # report id
    "tactical": 326,
  }
```

Transform and Load

This can be done using the pipeline framework backfill functionality.

Gitlab Code repository reference

https://gitlab.com/omg-technology/phd-global/data-connections/planit_new_report